Technical Design Document: DNS Migration from Unbound EC2 to AWS Route 53

1. Overview

1.1 Background

The current DNS infrastructure relies on a single self-managed Unbound DNS resolver running on an EC2 instance. This architecture presents a critical **single point of failure (SPOF)** that poses significant availability risks for users in Nepal accessing our application services.

1.2 Purpose

This document outlines the technical design for migrating from the self-managed Unbound DNS server to **AWS Route 53**, a fully managed, highly available, and scalable DNS service. The migration introduces:

- Redundant DNS resolution with 100% SLA
- Automated health-check-driven failover
- Latency-optimized routing for users in Nepal
- Reduced operational overhead

1.3 Scope

This document covers:

- DNS infrastructure migration only
- Route 53 hosted zone configuration
- Health check and failover setup
- Monitoring and alerting integration

Out of Scope:

- Application infrastructure changes
- Multi-region application deployment
- Domain registrar migration

---

2. Goals and Non-Goals

2.1 Goals

Goal	Description	Success Metric
Eliminate SPOF	Remove a single point of failure in DNS resolution	Zero DNS-related outages due to infrastructure failure
Automatic Failover	Implement health-check-driven failover to the maintenance page	Failover completes within ~90 seconds
Low Latency	Optimize DNS resolution for Nepal-based users	DNS resolution latency < 50ms
Reduce Operational Overhead	Eliminate EC2 patching and maintenance burden	Zero DNS infrastructure maintenance tasks
Cost Optimization	Reduce infrastructure costs	Monthly cost ~$1.50–$2.00

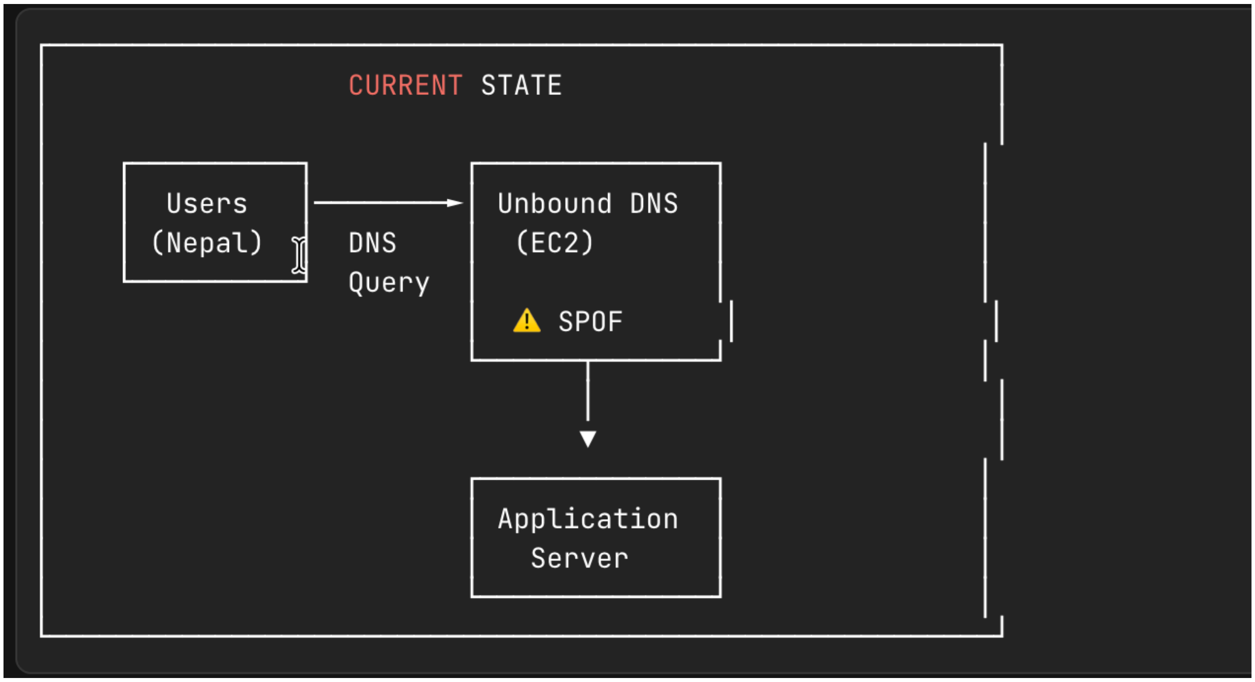
2.2 Non-Goals

- Migration of application servers or databases
- Implementation of multi-region active-active deployment
- Changes to application code or APIs
- DNSSEC implementation (future consideration)

---

3. Architecture

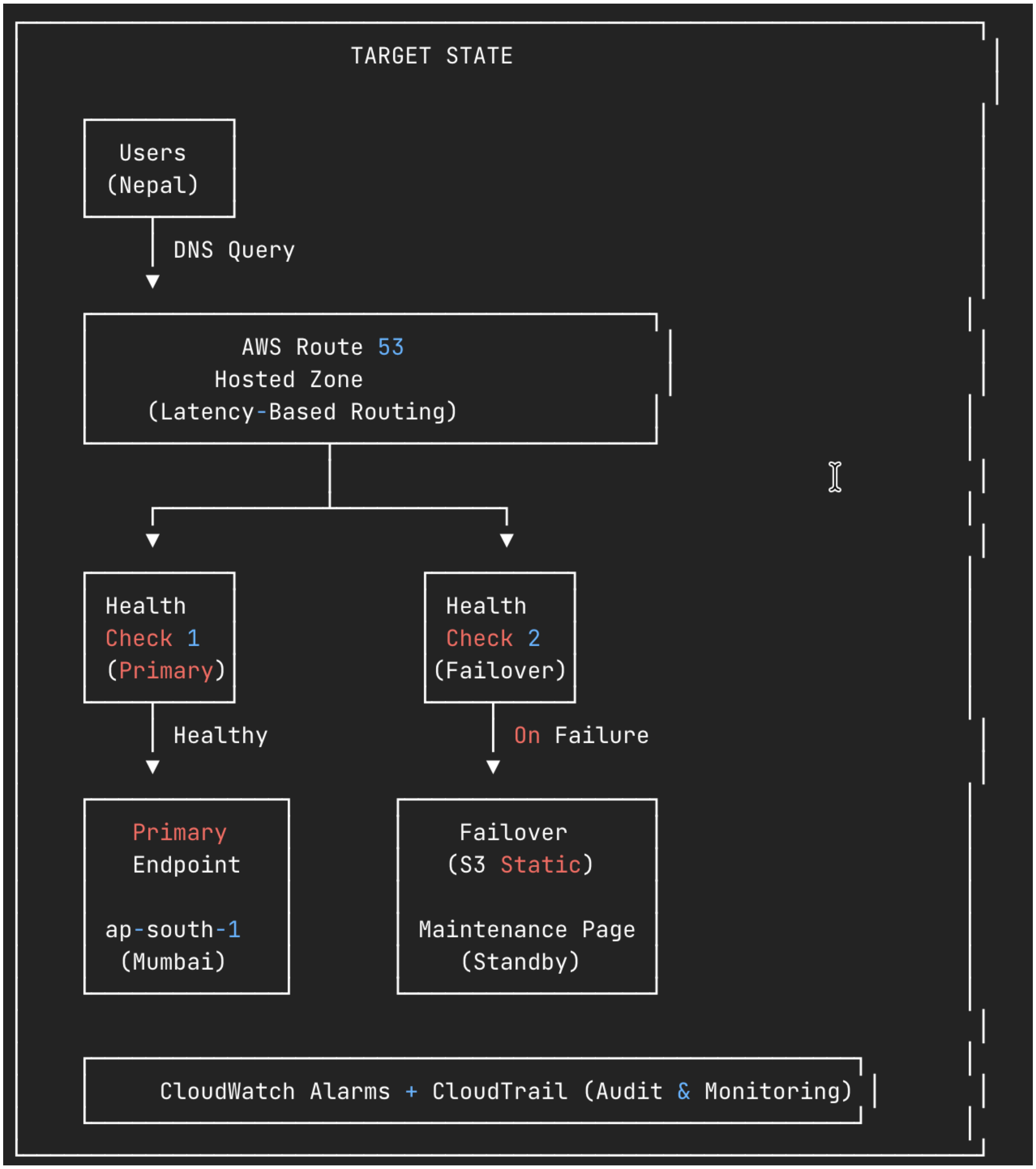
3.1 Current Architecture (Unbound EC2)



Current State Issues:

- **Single Point of Failure:** One EC2 instance handles all DNS queries
- **No Automatic Failover:** Manual intervention required during outages
- **Operational Overhead:** Requires patching, monitoring, and maintenance
- **No Health Checking:** No proactive detection of failures
- **No Geographic Optimization:** No latency-based routing

3.2 Target Architecture (AWS Route 53)



3.3 Component Overview

Component	Service	Purpose
Hosted Zone	Route 53	Managed DNS zone for domain records

Latency-Based Routing	Route 53	Routes users to the lowest-latency endpoint
Health Check (Primary)	Route 53	Monitors the primary application endpoint
Health Check (Failover)	Route 53	Monitors failover endpoint availability
Primary Endpoint	EC2/ELB (ap-south-1)	Main application server in Mumbai
Failover Endpoint	S3 Static Website	Maintenance page for outage scenarios
Monitoring	CloudWatch + CloudTrail	Metrics, alarms, and audit logging

3.4 Traffic Flow

1. **User Request:** User in Nepal initiates DNS query for app.example.com
2. **Route 53 Resolution:** Query reaches Route 53 hosted zone
3. **Routing Policy Evaluation:** Latency-based routing selects optimal endpoint
4. **Health Check Validation:** Route 53 verifies primary endpoint health status
5. **Response:**
    - **If Healthy:** Returns primary endpoint IP (ap-south-1)
    - **If Unhealthy:** Returns failover endpoint (S3 maintenance page)
6. **Connection:** User connects to resolved endpoint

3.5 Key Benefits

Benefit	**Description**

Eliminates SPOF	Fully managed, globally redundant DNS infrastructure
Automatic Failover	Health-check-driven failover (~90 seconds)
Low Latency	ap-south-1 (Mumbai) provides ~20–40ms latency to Nepal
Reduced Ops	No EC2 patching; costs ~$1.50–$2.00/month

---

4. Data Model

4.1 Route 53 Hosted Zone

HostedZone:
Name: example.com
Type: Public
Comment: "Production DNS zone - migrated from Unbound"

4.2 DNS Record Configuration

Primary Record (Latency-Based)

ResourceRecordSet:
Name: app.example.com
Type: A
SetIdentifier: primary-mumbai
Region: ap-south-1
TTL: 60
HealthCheckId: <primary-health-check-id>
ResourceRecords:
  - Value: <primary-endpoint-ip>

Failover Record (Secondary)

ResourceRecordSet:
Name: app.example.com
Type: A
SetIdentifier: failover-maintenance
Failover: SECONDARY

```
TTL: 60
AliasTarget:
  HostedZoneId: <s3-website-hosted-zone-id>
  DNSName: <s3-website-endpoint>
  EvaluateTargetHealth: false
```

4.3 Health Check Configuration

Primary Health Check

```
HealthCheck:
Name: primary-endpoint-health
Type: HTTPS
FullyQualifiedDomainName: app.example.com
Port: 443
ResourcePath: /health
RequestInterval: 30
FailureThreshold: 3
EnableSNI: true
Regions:
  - ap-south-1       # Mumbai (closest)
  - ap-southeast-1   # Singapore
  - us-east-1        # N. Virginia (global coverage)
```

4.4 CloudWatch Alarm Configuration

```
CloudWatchAlarm:
AlarmName: Route53-HealthCheck-Failed
Namespace: AWS/Route53
MetricName: HealthCheckStatus
Dimensions:
  - Name: HealthCheckId
    Value: <health-check-id>
Statistic: Minimum
Period: 60
EvaluationPeriods: 1
Threshold: 1
ComparisonOperator: LessThanThreshold
AlarmActions:
  - <sns-topic-arn>
OKActions:
  - <sns-topic-arn>
```

---

5. API Design

5.1 Health Check Endpoint Requirements

The application must expose a health check endpoint for Route 53 monitoring:

Endpoint Specification

```
GET /health
Host: app.example.com
```

Success Response (HTTP 200)


```
{
  "status": "healthy",
  "timestamp": "2024-01-15T10:30:00Z",
  "checks": {
    "database": "ok",
    "cache": "ok"
  }
}
```


Failure Response (HTTP 503)


```
{
  "status": "unhealthy",
  "timestamp": "2024-01-15T10:30:00Z",
  "checks": {
    "database": "failed",
    "cache": "ok"
  }
}
```


Health Check Requirements

Requirement	**Value**
Response Time	< 4 seconds
HTTP Status (Healthy)	2xx or 3xx
Content Type	application/json

Authentication	None (public endpoint)

5.2 Infrastructure as Code (Terraform)

```
# Route 53 Hosted Zone
resource "aws_route53_zone" "main" {
  name    = "example.com"
  comment = "Production DNS zone"
}

# Health Check
resource "aws_route53_health_check" "primary" {
  fqdn              = "app.example.com"
  port              = 443
  type              = "HTTPS"
  resource_path     = "/health"
  failure_threshold = 3
  request_interval  = 30

  tags = {
    Name = "primary-endpoint-health"
  }
}

# Primary Record
resource "aws_route53_record" "primary" {
  zone_id         = aws_route53_zone.main.zone_id
  name            = "app.example.com"
  type            = "A"
  ttl             = 60
  set_identifier  = "primary-mumbai"

  latency_routing_policy {
    region = "ap-south-1"
  }

  health_check_id = aws_route53_health_check.primary.id
  records         = [var.primary_endpoint_ip]
}
```

6. Security Considerations

6.1 Access Control

Control	Implementation	Priority
IAM Policies	Restrict Route 53 modifications to authorized roles	High
MFA Enforcement	Require MFA for DNS record changes	High
Audit Logging	Enable CloudTrail for all Route 53 API calls	High
Least Privilege	Create dedicated IAM roles for DNS management	Medium

6.2 IAM Policy Example

```
{
 "Version": "2012-10-17",
 "Statement": [
   {
    "Sid": "Route53DNSManagement",
    "Effect": "Allow",
    "Action": [
      "route53:GetHostedZone",
      "route53:ListResourceRecordSets",
      "route53:ChangeResourceRecordSets",
      "route53:GetHealthCheck",
      "route53:UpdateHealthCheck"
    ],
    "Resource": [
      "arn:aws:route53:::hostedzone/<zone-id>",
      "arn:aws:route53:::healthcheck/*"
    ],
    "Condition": {
      "Bool": {
        "aws:MultiFactorAuthPresent": "true"
```

```
      }
    }
  }
 ]
}
```

6.3 DNS Security

- **Query Logging:** Enable Route 53 query logging to CloudWatch for security analysis
- **DNSSEC:** Consider future implementation for DNS response authentication
- **Rate Limiting:** Whitelist Route 53 health checker IPs from WAF rate limits

6.4 Health Check Security

- **Minimal Response:** Health endpoint should not expose sensitive system information
- **IP Whitelisting:** Consider restricting `/health` to Route 53 health checker IP ranges
- **No Authentication:** Health check endpoint must be publicly accessible

---

7. Testing Strategy

7.1 Pre-Migration Testing

Test Case	**Description**	**Expected Result**
DNS Resolution	Query hosted zone directly	Correct IP returned
Health Check Status	Verify health check passes	Status: Healthy
TTL Verification	Confirm TTL configuration	TTL = 60 seconds
Latency Measurement	Test from Nepal region	< 50ms resolution

7.2 Failover Testing

Test Case	Procedure	Expected Result	Recovery Time
Primary Failure	Disable primary health check	Traffic routes to S3 failover	< 90 seconds
Primary Recovery	Re-enable primary health check	Traffic returns to primary	< 90 seconds
Endpoint Timeout	Simulate slow response (>4s)	Health check fails	~90 seconds

7.3 Testing Commands

```
# DNS Resolution Test
dig app.example.com @ns-xxx.awsdns-xx.com +short

# Latency Test
dig app.example.com | grep "Query time"

# Failover Simulation
aws route53 update-health-check \
  --health-check-id <id> \
  --disabled

# Monitor Resolution During Failover
watch -n 5 'dig +short app.example.com'
```

7.4 Test Sign-Off Checklist

- DNS resolution returns correct primary IP
- Health checks passing from all configured regions
- Failover completes within 90 seconds
- Recovery completes within 90 seconds
- CloudWatch alarms trigger on failure
- Query logging captures requests

8. Rollout Plan

8.1 Timeline Overview

Phase	**Duration**	**Activities**
Phase 1: Preparation	30 min	Create hosted zone, configure records
Phase 2: Health Checks	30 min	Create and validate health checks
Phase 3: Migration	30 min	Update nameservers, verify propagation
Phase 4: Testing	30 min	Execute failover tests
Phase 5: Monitoring	60 min	Observe production traffic
Total	**~3 hours**	

8.2 Detailed Steps

Phase 1: Preparation (30 minutes)

1. Create Route 53 hosted zone
2. Configure primary DNS record with latency-based routing
3. Configure failover DNS record pointing to S3
4. Set up S3 static maintenance page

Phase 2: Health Check Setup (30 minutes)

1. Create health check for primary endpoint
2. Verify health check status is "Healthy"
3. Associate health check with primary DNS record

4. Configure CloudWatch alarms

Phase 3: Migration (30 minutes)

1. Lower TTL on existing records (if not already 60s)
2. Update domain nameservers to Route 53
3. Monitor DNS propagation
4. Verify resolution from Nepal region

Phase 4: Testing (30 minutes)

1. Execute DNS resolution tests
2. Perform controlled failover test
3. Verify automatic recovery
4. Document results

Phase 5: Monitoring (60 minutes)

1. Monitor Route 53 query metrics
2. Verify health check stability
3. Confirm latency improvements
4. Schedule Unbound EC2 decommissioning

8.3 Rollback Plan

Trigger Conditions:

- DNS resolution failures > 5% of requests
- Health checks failing without endpoint issues
- Latency significantly degraded

Rollback Steps:

1. Revert nameservers to original configuration at registrar
2. Unbound EC2 instance remains operational during migration window
3. DNS propagates within TTL period (~60 seconds)
4. Document incident for post-mortem

8.4 Cost Summary

Component	**Monthly Cost**

Hosted Zone	$0.50
Health Checks (2)	$1.00
DNS Queries (~1M)	$0.40
Total	**~$1.50–$2.00**

Savings vs. Current: $13–$28/month (eliminating EC2 costs)

Appendix A: Region Selection

Region	Location	Latency to Nepal	Recommendation
ap-south-1	Mumbai	~20–40ms	**Primary**
ap-southeast-1	Singapore	~60–80ms	Secondary
ap-northeast-1	Tokyo	~100–120ms	Not recommended